

Text as Data
ifo Institute, 15–17 September 2026
Benjamin W. Arolt, University of Cambridge

8. LLMs and Transformers

Outline

Neural Language Models

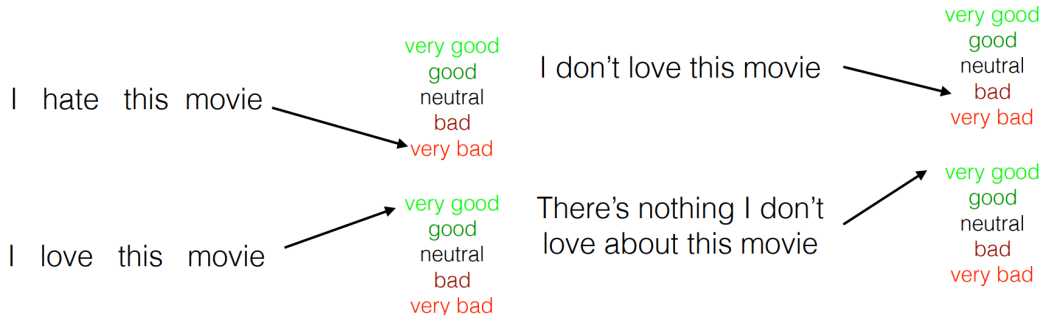
Transformer Models

BERT

GPT

From Language Model to Assistant: Training LLMs

The Classic Sentence Classification Problem



Source: Graham Neubig slides.

- ▶ bag-of-words models won't capture the importance of “don't love” or “nothing I don't love”, even with interactions / hidden layers.
- ▶ N-grams have a large feature space (especially with 4-grams) and don't share information across similar words/n-grams.

Sequence Data

- ▶ The real break-through from deep learning for NLP:
 - ▶ moving from bag-of-X representations to sequence representations.
 - ▶ Rather than inputting **counts over words** $\mathbf{x}$, take as input a **sequence of tokens** $\{w_1, \dots, w_t, \dots w_n\}$.

Sequence Data

- ▶ The real break-through from deep learning for NLP:
 - ▶ moving from bag-of-X representations to sequence representations.
 - ▶ Rather than inputting **counts over words** $\mathbf{x}$, take as input a **sequence of tokens** $\{w_1, \dots, w_t, \dots, w_n\}$.
 - ▶ Goal: Predict the next word in a sequence given all previous words (left) or given N-1 previous words (right):

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-N+1}, \dots, w_{t-1})$$

Sequence Data

- ▶ The real break-through from deep learning for NLP:
 - ▶ moving from bag-of-X representations to sequence representations.
 - ▶ Rather than inputting **counts over words** $\mathbf{x}$, take as input a **sequence of tokens** $\{w_1, \dots, w_t, \dots, w_n\}$.
 - ▶ Goal: Predict the next word in a sequence given all previous words (left) or given N-1 previous words (right):

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-N+1}, \dots, w_{t-1})$$

- ▶ Self-supervised ML with text (typically no pre-processing).

Sequence Data

- ▶ The real break-through from deep learning for NLP:
 - ▶ moving from bag-of-X representations to sequence representations.
 - ▶ Rather than inputting **counts over words** $\mathbf{x}$, take as input a **sequence of tokens** $\{w_1, \dots, w_t, \dots, w_n\}$.
 - ▶ Goal: Predict the next word in a sequence given all previous words (left) or given N-1 previous words (right):

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-N+1}, \dots, w_{t-1})$$

- ▶ Self-supervised ML with text (typically no pre-processing).
- ▶ Use word embeddings instead of raw word identities.

Feedforward Neural Language Model

- ▶ **Input:** One-hot encoded word vectors from a fixed-size context window.
- ▶ **Example:** Given a vocabulary size of $|V|$:

dog = $[0, 0, 0, 0, 1, 0, \dots, V]$

cat = $[0, 1, 0, 0, 0, 0, \dots, V]$

Feedforward Neural Language Model

- ▶ **Input:** One-hot encoded word vectors from a fixed-size context window.
- ▶ **Example:** Given a vocabulary size of $|V|$:

$$\text{dog} = [0, 0, 0, 0, 1, 0, \dots, V]$$

$$\text{cat} = [0, 1, 0, 0, 0, 0, \dots, V]$$

- ▶ **Embedding Vector:** Converts one-hot vectors into dense embedding vectors.

$$\mathbf{v}_{\text{dog}} = W_{\text{embed}} \times [0, 0, 0, 0, 1, 0, \dots, V]^T$$

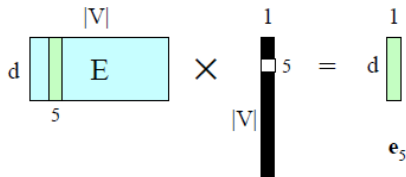


Figure: Selecting the embedding vector for word V_5 by multiplying the embedding matrix E with a one-hot vector with a 1 in index 5. (Jurafsky and Martin, 2024)

Feedforward Neural Language Model

- ▶ **Embedding Layer:** The m resulting embedding vectors are concatenated to produce e , the embedding layer (where m equals the context window)
- ▶ **Hidden Layer:** Uses activation functions (ReLU, tanh, etc.).
- ▶ **Output Layer:** Produces a probability distribution over possible next words using softmax.

Feedforward Neural Language Model: Example

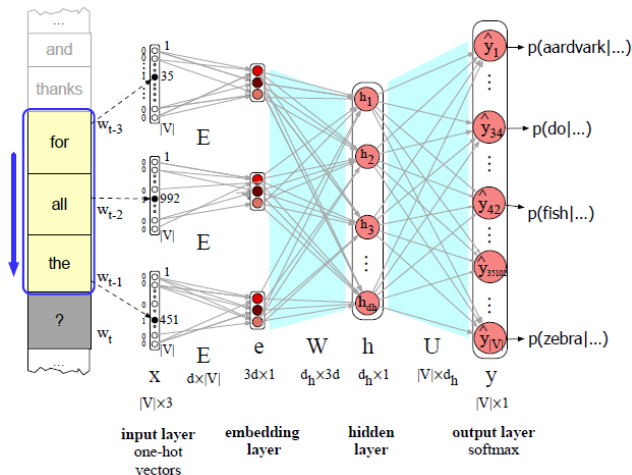


Figure: Forward inference in a feedforward neural language model. At each timestep t the network computes a d -dimensional embedding for each context word (by multiplying a one-hot vector by the embedding matrix E), and concatenates the 3 resulting embeddings to get the embedding layer e . The embedding vector e is multiplied by a weight matrix W and then an activation function is applied element-wise to produce the hidden layer h , which is then multiplied by another weight matrix U . Finally, a softmax output layer predicts at each node i the probability that the next word $\{w_t\}$ will be vocabulary word $\{V_i\}$. (Jurafsky and Martin, 2024)

Training the Neural Language Model

- ▶ **Loss function:** Cross-entropy loss for classification:

$$L_{CE}(\hat{y}, y) = -\log \hat{y}_i = -\log p(w_t \mid w_{t-1}, \dots, w_{t-n+1}), \quad (\text{where } i \text{ is the correct class})$$

where:

- ▶ y_i is the true label (one-hot encoding).
 - ▶ $\hat{y}_i$ is the predicted probability from the softmax layer.
- ▶ **Optimization:** Uses gradient descent and backpropagation to update parameters θ (embedding matrix, weight matrices, and biases):

$$\theta_{s+1} = \theta_s - \eta \frac{\partial [-\log p(w_t \mid w_{t-1}, \dots, w_{t-n+1})]}{\partial \theta}$$

Challenges of Feedforward Neural Language Models

- ▶ **Challenges:**

- ▶ Computationally expensive to train and deploy.
- ▶ Requires large datasets to perform well.
- ▶ Limited context window (compared to RNNs, transformers).
- ▶ Limited performance

Outline

Neural Language Models

Transformer Models

BERT

GPT

From Language Model to Assistant: Training LLMs

Extensions of Feedforward Neural Language Models

Neural language models have evolved to address limitations of feedforward networks:

- ▶ **Recurrent Neural Networks (RNNs)**
 - ▶ Introduce recurrence (hidden states) to handle sequential data
 - ▶ Capture long-range dependencies, but suffer from vanishing gradients.
 - ▶ More in lecture on audio data.

Extensions of Feedforward Neural Language Models

Neural language models have evolved to address limitations of feedforward networks:

- ▶ **Recurrent Neural Networks (RNNs)**

- ▶ Introduce recurrence (hidden states) to handle sequential data
- ▶ Capture long-range dependencies, but suffer from vanishing gradients.
- ▶ More in lecture on audio data.

- ▶ **Convolutional Neural Networks (CNNs)**

- ▶ Use convolutional filters to extract local features.
- ▶ Efficient due to parallel processing; struggles with long-range dependencies.
- ▶ More in lecture on image data.

Extensions of Feedforward Neural Language Models

Neural language models have evolved to address limitations of feedforward networks:

- ▶ **Recurrent Neural Networks (RNNs)**

- ▶ Introduce recurrence (hidden states) to handle sequential data
- ▶ Capture long-range dependencies, but suffer from vanishing gradients.
- ▶ More in lecture on audio data.

- ▶ **Convolutional Neural Networks (CNNs)**

- ▶ Use convolutional filters to extract local features.
- ▶ Efficient due to parallel processing; struggles with long-range dependencies.
- ▶ More in lecture on image data.

- ▶ **Transformer Models**

- ▶ Use self-attention for global context.
- ▶ State-of-the-art in text data analysis
- ▶ Examples: BERT, GPT.

Transformers

- ▶ Since a 2017 paper (Vaswani et al 2017), deep learning for NLP has been transformed by a new class of models: **transformers**.

Transformers

- ▶ Since a 2017 paper (Vaswani et al 2017), deep learning for NLP has been transformed by a new class of models: **transformers**.
- ▶ Standard approach:
 - ▶ represent documents as counts over words/phrases, shares over topics, or the average of word embeddings.

Transformers

- ▶ Since a 2017 paper (Vaswani et al 2017), deep learning for NLP has been transformed by a new class of models: **transformers**.
- ▶ Standard approach:
 - ▶ represent documents as counts over words/phrases, shares over topics, or the average of word embeddings.
- ▶ Recurrent neural nets can process whole documents word-by-word:
 - ▶ but they have to sweep through the whole document at each training epoch, so they learn too slowly.

Transformers

- ▶ Since a 2017 paper (Vaswani et al 2017), deep learning for NLP has been transformed by a new class of models: **transformers**.
- ▶ Standard approach:
 - ▶ represent documents as counts over words/phrases, shares over topics, or the average of word embeddings.
- ▶ Recurrent neural nets can process whole documents word-by-word:
 - ▶ but they have to sweep through the whole document at each training epoch, so they learn too slowly.
- ▶ Transformers overcome these limitations:
 - ▶ intuitively, they provide a way to efficiently read in an entire document and learn the meaning of all words and all interactions between words.

Self-Attention – the fundamental computation underlying transformers

- ▶ Consider a sequence of tokens with fixed length n_L , $\{w_1, \dots, w_i, \dots, w_{n_L}\}$
- ▶ We have word embedding vectors $x_i = E(w_i)$ with dimension n_E , producing a sequence of vectors

$$\{x_1, \dots, x_i, \dots, x_{n_L}\}$$

- ▶ In previous models, the sequence $x_{1:n_L}$ could be flattened to an $n_L n_E$ -dimensional vector and piped to the hidden layers for use in the task, e.g. sentiment classification.

Self-Attention – the fundamental computation underlying transformers

- ▶ Consider a sequence of tokens with fixed length n_L , $\{w_1, \dots, w_i, \dots, w_{n_L}\}$
- ▶ We have word embedding vectors $x_i = E(w_i)$ with dimension n_E , producing a sequence of vectors

$$\{x_1, \dots, x_i, \dots, x_{n_L}\}$$

- ▶ In previous models, the sequence $x_{1:n_L}$ could be flattened to an $n_L n_E$ -dimensional vector and piped to the hidden layers for use in the task, e.g. sentiment classification.
- ▶ A **self-attention layer** transforms $x_{1:n_L}$ into a second sequence $h_{1:n_L}$, where

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

- ▶ where $a(\cdot)$ is an attention function such that $a(\cdot) \geq 0$, $\sum a(\cdot) = 1$.
 - ▶ $\rightarrow$ each h_i becomes a weighted average of the whole sequence.
- ▶ $h_{1:n_L}$ is flattened and piped to the network's hidden layers, rather than $x_{1:n_L}$.

Basic Self-Attention

Setup:

1. Sequence of tokens $\{w_1, \dots, w_i, \dots, w_{n_L}\}$
2. Sequence of (trainable) embedding vectors $\{x_1, \dots, x_i, \dots, x_{n_L}\}$
3. Sequence of attention-transformed vectors $\{h_1, \dots, h_i, \dots, h_{n_L}\}$ with

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

Basic Self-Attention

Setup:

1. Sequence of tokens $\{w_1, \dots, w_i, \dots, w_{n_L}\}$
2. Sequence of (trainable) embedding vectors $\{x_1, \dots, x_i, \dots, x_{n_L}\}$
3. Sequence of attention-transformed vectors $\{h_1, \dots, h_i, \dots, h_{n_L}\}$ with

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

Basic self-attention specifies

$$a(x_i, x_j) = \frac{\exp(x_i \cdot x_j)}{\sum_{k=1}^{n_L} \exp(x_i \cdot x_k)}$$

- the dot-product $x_i \cdot x_j$, normalized with softmax such that $\sum_j a(\cdot) = 1$.

Basic Self-Attention

Setup:

1. Sequence of tokens $\{w_1, \dots, w_i, \dots, w_{n_L}\}$
2. Sequence of (trainable) embedding vectors $\{x_1, \dots, x_i, \dots, x_{n_L}\}$
3. Sequence of attention-transformed vectors $\{h_1, \dots, h_i, \dots, h_{n_L}\}$ with

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

Basic self-attention specifies

$$a(x_i, x_j) = \frac{\exp(x_i \cdot x_j)}{\sum_{k=1}^{n_L} \exp(x_i \cdot x_k)}$$

- ▶ the dot-product $x_i \cdot x_j$, normalized with softmax such that $\sum_j a(\cdot) = 1$.
- ▶ Putting it together:

$$h_i = \sum_{j=1}^{n_L} \frac{\exp(x_i \cdot x_j)}{\sum_{k=1}^{n_L} \exp(x_i \cdot x_k)} x_j$$

- The basic self-attention transformation

$$h_i = \sum_{j=1}^{n_L} \frac{\exp(x_i \cdot x_j)}{\sum_{k=1}^{n_L} \exp(x_i \cdot x_k)} x_j$$

is the foundational ingredient of transformers.

- ▶ The basic self-attention transformation

$$h_i = \sum_{j=1}^{n_L} \frac{\exp(x_i \cdot x_j)}{\sum_{k=1}^{n_L} \exp(x_i \cdot x_k)} x_j$$

is the foundational ingredient of transformers.

Note the following simplifications:

- ▶ basic self-attention has no learnable parameters.
 - ▶ self-attention works indirectly through allowing the word embeddings to interact with each other
- ▶ basic self-attention ignores word order.

- ▶ The basic self-attention transformation

$$h_i = \sum_{j=1}^{n_L} \frac{\exp(x_i \cdot x_j)}{\sum_{k=1}^{n_L} \exp(x_i \cdot x_k)} x_j$$

is the foundational ingredient of transformers.

Note the following simplifications:

- ▶ basic self-attention has no learnable parameters.
 - ▶ self-attention works indirectly through allowing the word embeddings to interact with each other
- ▶ basic self-attention ignores word order.

The big initial gain from transformers, relative to RNNs, came from basic self-attention.

- ▶ The successful models (e.g. BERT, GPT) do **add parameters and word order information** to $a(\cdot)$

Self-attention allows words to interact with each other

- Consider a sentence

the, cat, walks, on, the, street

with embeddings

$\mathbf{x}_{\text{the}}, \mathbf{x}_{\text{cat}}, \mathbf{x}_{\text{walks}}, \mathbf{x}_{\text{on}}, \mathbf{x}_{\text{the}}, \mathbf{x}_{\text{street}}$

- Feeding this sentence into the self-attention layer produces

$\mathbf{h}_{\text{the}}, \mathbf{h}_{\text{cat}}, \mathbf{h}_{\text{walks}}, \mathbf{h}_{\text{on}}, \mathbf{h}_{\text{the}}, \mathbf{h}_{\text{street}}$

where $\mathbf{h}_i = \sum_{j=1}^n \frac{\exp(\mathbf{x}_i \cdot \mathbf{x}_j)}{\sum_k \exp(\mathbf{x}_i \cdot \mathbf{x}_k)} \cdot \mathbf{x}_j$.

Self-attention allows words to interact with each other

- ▶ Consider a sentence

the, cat, walks, on, the, street

with embeddings

$\mathbf{x}_{\text{the}}, \mathbf{x}_{\text{cat}}, \mathbf{x}_{\text{walks}}, \mathbf{x}_{\text{on}}, \mathbf{x}_{\text{the}}, \mathbf{x}_{\text{street}}$

- ▶ Feeding this sentence into the self-attention layer produces

$\mathbf{h}_{\text{the}}, \mathbf{h}_{\text{cat}}, \mathbf{h}_{\text{walks}}, \mathbf{h}_{\text{on}}, \mathbf{h}_{\text{the}}, \mathbf{h}_{\text{street}}$

where $\mathbf{h}_i = \sum_{j=1}^n \frac{\exp(\mathbf{x}_i \cdot \mathbf{x}_j)}{\sum_k \exp(\mathbf{x}_i \cdot \mathbf{x}_k)} \cdot \mathbf{x}_j$.

Embedding layer will learn vectors $\mathbf{x}$ that tend to have **attention dot products** that contribute to the task at hand.

- ▶ For example, most transformers are pre-trained on a language modeling task (predicting a left-out word or sentence)
- ▶ in this task, stopwords like “the” will not be helpful.
 - ▶ the learned embedding $\mathbf{x}_{\text{the}}$ will tend to have a low or negative dot product with more informative words.

Adding Parameters: Query, Key, and Value

- ▶ In *basic* self-attention, each token uses the *same* vector x_i for every role – and there are no learnable parameters
- ▶ Idea: give each token three learned **roles**, via three weight matrices W_Q, W_K, W_V :

Query	$q_i = W_Q x_i$	“what I am looking for”
Key	$k_i = W_K x_i$	“what I offer to others”
Value	$v_i = W_V x_i$	“what I pass on if attended to”

Adding Parameters: Query, Key, and Value

- ▶ In *basic* self-attention, each token uses the *same* vector x_i for every role – and there are no learnable parameters
- ▶ Idea: give each token three learned **roles**, via three weight matrices W_Q, W_K, W_V :

Query $q_i = W_Q x_i$ “what I am looking for”

Key $k_i = W_K x_i$ “what I offer to others”

Value $v_i = W_V x_i$ “what I pass on if attended to”

- ▶ Attention from token i to token j compares i 's *query* with j 's *key*:

$$a(x_i, x_j) = \frac{\exp(q_i \cdot k_j / \sqrt{d_k})}{\sum_{l=1}^{n_L} \exp(q_i \cdot k_l / \sqrt{d_k})}$$

- ▶ dividing by $\sqrt{d_k}$ (d_k = dimension of queries/keys) keeps the dot products from growing too large
- ▶ The new vector is a weighted average of the *values*:

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) v_j$$

Adding Parameters: Further Notes

- ▶ **Learnable:** W_Q, W_K, W_V are trained – basic self-attention is just the special case $W_Q = W_K = W_V = I$
- ▶ **Multi-head attention:** run several (W_Q, W_K, W_V) in parallel; each “head” can learn a different relation (e.g. subject–verb, coreference), then concatenate
 - ▶ this is the core building block inside BERT and GPT

Autoregressive vs Autoencoding Language Models

▶ Autoregressive models:

- ▶ e.g. **GPT** = “**Generative Pre-Trained Transformer**”:
- ▶ pretrained on classic language modeling task: guess the next token having read all the previous ones.
- ▶ during training, attention heads only view previous tokens, not subsequent tokens.
- ▶ ideal for text generation.

▶ Autoencoding models

- ▶ e.g. **BERT** = “**Bidirectional Encoder Representations from Transformers**”
- ▶ pretrained by dropping/shuffling input tokens and trying to reconstruct the original sequence.
- ▶ usually build bidirectional representations and get access to the full sequence.
- ▶ can be fine-tuned and achieve great results on many tasks, e.g. text classification.

Outline

Neural Language Models

Transformer Models

BERT

GPT

From Language Model to Assistant: Training LLMs

Autoencoding Language Models

- ▶ **Main example: BERT**

- ▶ e.g. **BERT = “Bidirectional Encoder Representations from Transformers”**
- ▶ pretrained by dropping/shuffling input tokens and trying to reconstruct the original sequence.
- ▶ usually build bidirectional representations and get access to the full sequence.
- ▶ can be fine-tuned and achieve great results on many tasks, e.g. text classification.

BERT I/II

- ▶ Task: Masked language modeling:
 - ▶ 15% of words masked
 - ▶ if masked: replace with [MASK] 80% of the time, a random token 10% of the time, and left unchanged 10% of the time.
 - ▶ model has to predict the original word.
- ▶ Architecture:
 - ▶ Architecture:
 - ▶ A stack of Transformer blocks, each containing a multi-head self-attention layer and an MLP.
 - ▶ BERT-Base has 12 blocks, a hidden dimension of 768, and 12 attention heads: approximately 110M parameters.
 - ▶ BERT-Large has 24 blocks, a hidden dimension of 1,024, and 16 attention heads: approximately 340M parameters.

BERT II/II

- ▶ Unlike GPT, BERT attention observes all tokens in the sequence, reads backwards and forwards (bidirectional).
- ▶ Corpus:
 - ▶ 800M words from English books (modern work, from unpublished authors)
 - ▶ 2.5B words of text from English Wikipedia articles
- ▶ Various extensions:
 - ▶ RoBERTa: improves BERT's pre-training procedure.
 - ▶ DistilBERT: a smaller and faster version of BERT.
 - ▶ VisualBERT: jointly processes text and image-region features.

Outline

Neural Language Models

Transformer Models

BERT

GPT

From Language Model to Assistant: Training LLMs

Autoregressive Language Models

- ▶ **Main example: GPT**

- ▶ e.g. **GPT** = “**Generative Pre-Trained Transformer**”:
- ▶ pretrained on classic language modeling task: guess the next token having read all the previous ones.
- ▶ during training, attention heads only view previous tokens, not subsequent tokens.
- ▶ ideal for text generation.

GPT = Generative Pre-Trained Transformer

GPT = Generative Pre-Trained Transformer

- ▶ GPT-1: the first autoregressive transformer model (2018)
 - ▶ trained on the Books corpus (7000+ books)
 - ▶ train on a language modeling task, as well as a multi-task that adds a supervised learning task.

GPT = Generative Pre-Trained Transformer

- ▶ GPT-1: the first autoregressive transformer model (2018)
 - ▶ trained on the Books corpus (7000+ books)
 - ▶ train on a language modeling task, as well as a multi-task that adds a supervised learning task.
- ▶ GPT-2 (2019):
 - ▶ all articles linked from Reddit with at least 3 upvotes (8 million documents, 40 GB of text)
 - ▶ dispense with supervised learning task, make some other architectural adjustments
 - ▶ make model much bigger

GPT = Generative Pre-Trained Transformer

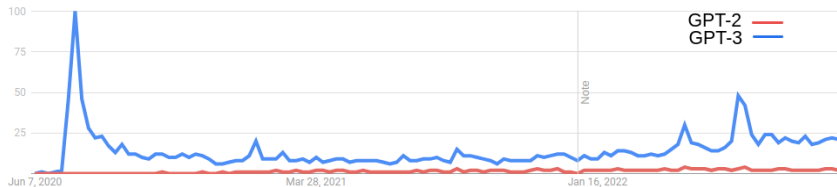
- ▶ GPT-1: the first autoregressive transformer model (2018)
 - ▶ trained on the Books corpus (7000+ books)
 - ▶ train on a language modeling task, as well as a multi-task that adds a supervised learning task.
- ▶ GPT-2 (2019):
 - ▶ all articles linked from Reddit with at least 3 upvotes (8 million documents, 40 GB of text)
 - ▶ dispense with supervised learning task, make some other architectural adjustments
 - ▶ make model much bigger
- ▶ GPT-3 (2020):
 - ▶ use an even bigger corpus (Common Crawl, WebText2, Books1, Books2 and Wikipedia)
 - ▶ make model much, much bigger

OPENAI'S NEW MULTITALENTED AI WRITES, TRANSLATES, AND SLANDERS

A step forward in AI text-generation that also spells trouble

By James Vincent | Feb 14, 2019, 12:00pm EST

Howard, co-founder of Fast.AI agrees. "I've been trying to warn people about this for a while," he says. "We have the technology to totally fill Twitter, email, and the web up with reasonable-sounding, context-appropriate prose, which would drown out all other speech and be impossible to filter."



BUSINESS

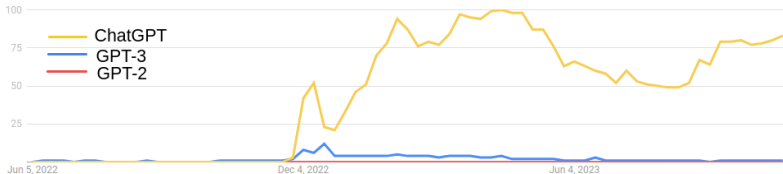
Is ChatGPT the Start of the AI Revolution?

Analysis by The Editors | Bloomberg

December 9, 2022 at 1:39 p.m. EST

The New Chatbots Could Change the World. Can You Trust Them?

Siri, Google Search, online marketing and your child's homework will never be the same. Then there's the misinformation problem.



GPT = Generative Pre-Trained Transformer

- ▶ GPT-1: the first autoregressive transformer model (2018)
- ▶ GPT-2 (2019)
- ▶ GPT-3 (2020)

GPT = Generative Pre-Trained Transformer

- ▶ GPT-1: the first autoregressive transformer model (2018)
- ▶ GPT-2 (2019)
- ▶ GPT-3 (2020)
- ▶ GPT-3.5 (2022):
 - ▶ Subclass of GPT-3 trained on data up to June 2021
 - ▶ Incorporates the base model on which ChatGPT is fine-tuned + is optimized for chat

GPT = Generative Pre-Trained Transformer

- ▶ GPT-1: the first autoregressive transformer model (2018)
- ▶ GPT-2 (2019)
- ▶ GPT-3 (2020)
- ▶ GPT-3.5 (2022):
 - ▶ Subclass of GPT-3 trained on data up to June 2021
 - ▶ Incorporates the base model on which ChatGPT is fine-tuned + is optimized for chat
- ▶ GPT-4 (2023):
 - ▶ Multimodal model: can take also images as inputs
 - ▶ Trained in two stages:
 1. token prediction (like other GPT models)
 2. **reinforcement learning with human feedback**
 - ▶ Much, much, much bigger model

Outline

Neural Language Models

Transformer Models

BERT

GPT

From Language Model to Assistant: Training LLMs

From GPT to ChatGPT: what turns a language model into an assistant?

- ▶ **Type of generative AI that produces text**
- ▶ AI systems trained to predict the next word given preceding text
- ▶ Typically fine-tuned to follow human instructions and generate responses aligned with human preferences
- ▶ Based on deep neural networks with billions of parameters
- ▶ Built on transformer models (with attention mechanisms, which endogenously assign varying degrees of importance to different words)

Step 1: Pre-Training

- ▶ Calculate conditional probability distribution over words given the preceding words, based on its training data (**Next Token Prediction**)
- ▶ **Self-Supervised Learning**: Model is fed text fragments; parameters are adjusted to predict continuation
- ▶ Terabytes of data (Wikipedia, scientific articles, books, etc.)
- ▶ Neural nets learn language structure:
 - ▶ Syntactic structures
 - ▶ Relationships between words and concepts they represent
 - ▶ Context of sentences and word interactions
 - ▶ Relationships between different sentences

Step 2: Instruction Fine-Tuning

- ▶ Improves model to follow human instructions
- ▶ Example: Pretrained model might predict a continuation of “What is your name?” as “And how old are you?”; this is not what we want
- ▶ Instruction fine-tuning trains the model to respond appropriately
- ▶ **Supervised learning:** Feeding the model millions of examples for responding to thousands of different instructions
- ▶ Used for tasks like summarization, answering questions, brainstorming, etc.

Step 3: Reinforcement Learning

- ▶ Improves model by incorporating human feedback
- ▶ Human raters provide feedback on different responses
- ▶ Helps align model with human preferences, particularly in areas hard to define via instruction fine-tuning
- ▶ Example: Penalizing hateful responses
- ▶ Noisy process:
 - ▶ Partly why LLMs sound authoritative even when hallucinating